

Система автоматизированного тестирования и самопроверки знаний



ДПП ПП «Фронтенд и бэкенд разработка»

Выполнил: студент группы ИКБО-11-23
Дицель Н.С.

ВМЕСТЕ СОЗДАЁМ БУДУЩЕЕ

Цель и задачи работы

Цель

Разработать клиент-серверное веб-приложение для автоматизации создания, прохождения, проверки и анализа образовательных тестов.

- провести анализ предметной области и конкурентных решений;
- спроектировать роли пользователей, сущности и связи базы данных;
- реализовать React-интерфейс и REST API на Node.js/Express;
- добавить автоматическую проверку, статистику и экспорт отчётов;
- подготовить тестирование, Docker-среду, Nginx и CI/CD.



Результат: приложение готово к запуску

Анализ конкурентов

Проблема

- ручная проверка и перенос результатов занимают время;
- внешние сервисы ограничивают контроль данных и логики;
- для преподавателя важны роли, аналитика и повтор ошибок;
- нужна компактная система без избыточности полноценной LMS.

Критерий	Moodle	Google Forms	Quizizz	Проект
Роли и права	+	-	+/-	+
Собственная БД	+	-	-	+
Аналитика	+	+/-	+/-	+
Повтор ошибок	+/-	-	+/-	+
Простота внедрения	+/-	+	+	+

Итог выбора

Разработанная система объединяет простоту формы, ролевой контроль LMS и собственное хранение данных без зависимости от внешней платформы.

Требования к веб-приложению и его характеристика

Требования к веб-приложению

Ключевые функции

- регистрация, вход и профиль пользователя;
- роли: студент, преподаватель, администратор;
- создание тестов и вопросов single / multiple / matching;
- прохождение теста, расчёт балла и сохранение попытки;
- аналитика по результатам и экспорт PDF/Excel.

Нефункциональные требования

- JWT-авторизация и проверка ролей на сервере;
- bcrypt-хеширование паролей;
- валидация данных на frontend и backend;
- переменные окружения вместо секретов в коде;
- контейнеризация через Docker Compose и Nginx.

Студент

проходит тест
видит результат
повторяет ошибки

Преподаватель

создаёт тесты
редактирует вопросы
анализирует попытки

Администратор

управляет ролями
модерирует тесты
настраивает систему

Используемый для реализации проекта технологический стек

Frontend, backend, база данных и инфраструктура

Frontend

React, React Router, Redux Toolkit, Axios, CSS Modules

+

Backend

Node.js, Express.js, REST API, Sequelize ORM

+

Данные

PostgreSQL: users, tests, questions, test_results, system_settings

+

Инфраструктура

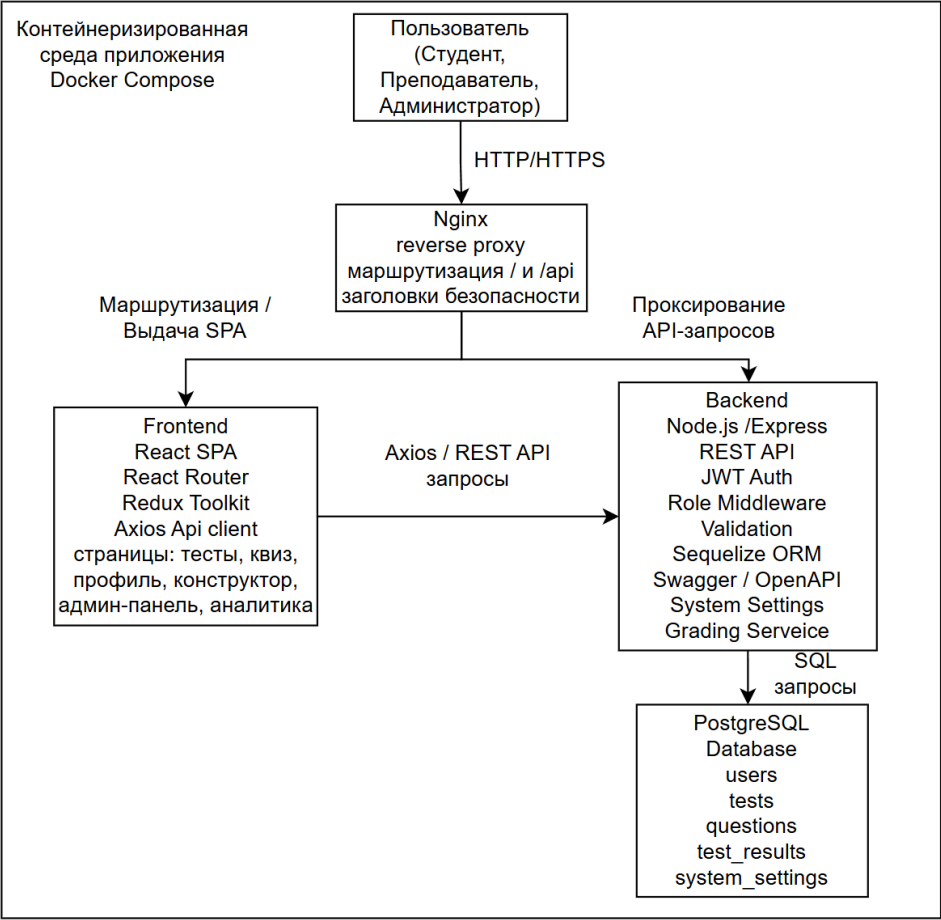
Docker Compose, Nginx, Render, GitHub Actions, Swagger/OpenAPI

Почему этот стек подходит

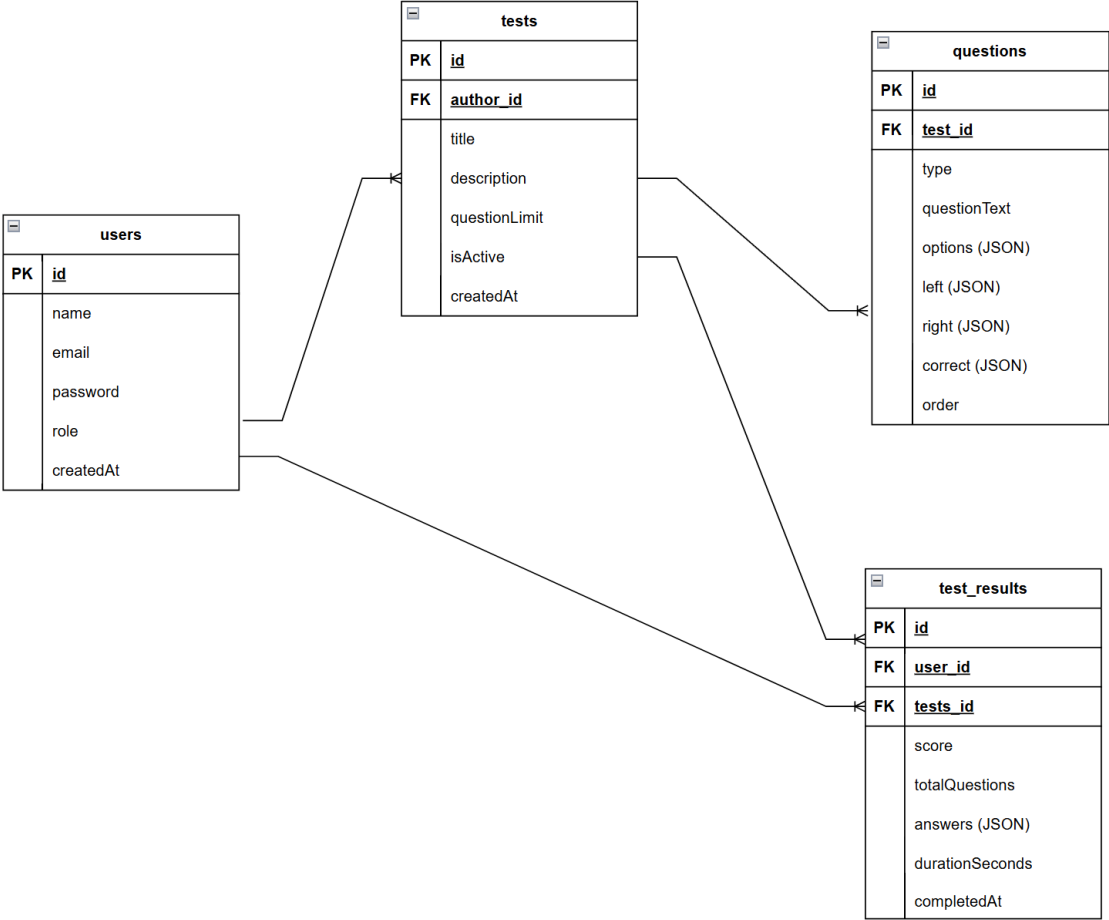
- React SPA быстро переключает разделы без перезагрузки страницы;
- Express REST API отделяет бизнес-логику от интерфейса;
- Sequelize описывает модели и связи с PostgreSQL;
- Swagger упрощает проверку API;
- Docker делает запуск переносимым.

PDF/Excel отчёты: pdfmake + xlsx

Защита: JWT + bcryptjs + express-validator



Общая архитектура



Сущности базы данных

От пользовательского сценария до сохранённого результата



Frontend

страницы Home, Login, Register, Test, Profile, TeacherAnalytics, AdminDashboard, QuestionsAdmin; состояние хранится в Redux slices.

Backend

модули auth, tests, results, admin; middleware авторизации, ролей и валидации; OpenAPI-документация.

Проверка ответов

нормализация вариантов, контроль структуры matching-вопросов, сохранение попытки и списка ошибочных вопросов.

82

Frontend tests

29

Backend tests

100%

coverage
statements/branches/functions/line
S

37

Fuzzing failed checks

```
PS C:\Users\relax\Documents\kursach_6_PiRKSP\client> npm run coverage
> test-constructor-client@1.0.0 coverage
> react-scripts test --coverage --watchAll=false --runInBand
```

File	% Stmts	% Branch	% Funcs	% Lines	Uncovered Line #s
All files	100	100	100	100	
src	100	100	100	100	
App.js	100	100	100	100	
src/components/Header	100	100	100	100	
Header.js	100	100	100	100	
src/components/Question	100	100	100	100	

```
PS C:\Users\relax\Documents\kursach_6_PiRKSP\server> npm run coverage
> test-constructor-server@1.0.0 coverage
> node ../client/node_modules/jest/bin/jest.js --coverage --runInBand
```

File	% Stmts	% Branch	% Funcs	% Lines	Uncovered Line #s
All files	100	100	100	100	
constants	100	100	100	100	
roles.js	100	100	100	100	
middleware	100	100	100	100	
auth.js	100	100	100	100	
optionalAuth.js	100	100	100	100	
role.js	100	100	100	100	
validate.js	100	100	100	100	
services	100	100	100	100	
systemSettings.js	100	100	100	100	

```
PASS Large Payload Register -> /api/auth/register (400)
PASS Large Payload Register -> /api/auth/register (400)
PASS Large Payload Register -> /api/auth/register (400)

Testing RBAC...

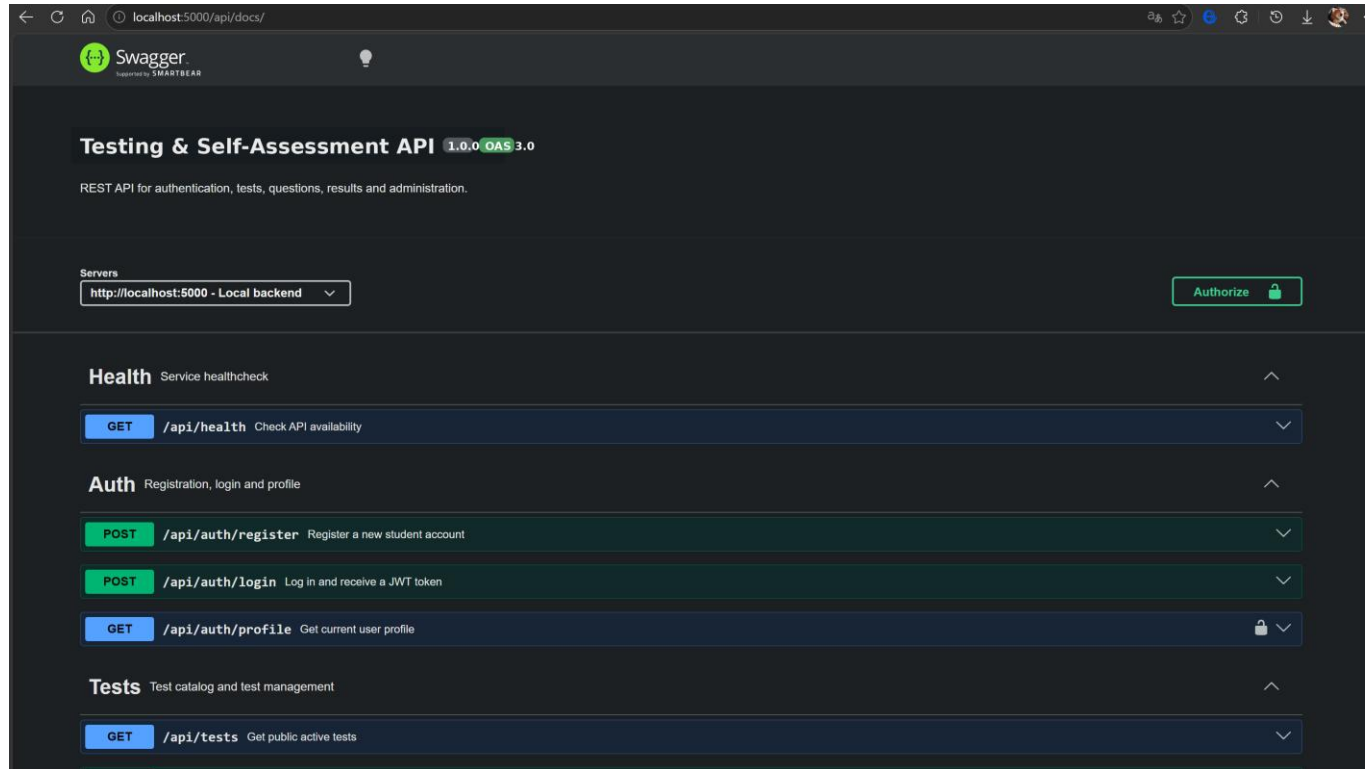
PASS Auth Login -> /api/auth/login (200)
PASS Auth Login -> /api/auth/login (200)
PASS Auth Login -> /api/auth/login (200)
PASS RBAC Create Test (admin) -> /api/tests (201)
PASS RBAC Delete Test (admin) -> /api/tests/42 (200)
PASS RBAC Create Test (teacher) -> /api/tests (201)
PASS RBAC Delete Test (teacher) -> /api/tests/43 (200)
PASS RBAC Create Test (student) -> /api/tests (403)

Results saved to: C:\Users\relax\Documents\kursach_6_PiRKSP\server
Passed checks: 37
Failed checks: 0
```

Проверены UI-компоненты, Redux-состояние, сервис API, middleware, роли, Swagger и устойчивость к SQL/XSS/Traversal payloads.

Демонстрация API и работоспособности

Swagger-документация, healthcheck и production-ссылки



```
PS C:\Users\relax\Documents\kursach_6_PIRKSP> curl http://localhost:5000/api/health
StatusCode      : 200
StatusDescription : OK
Content         : {"status":"OK","timestamp":"2026-05-04T19:43:28.482Z","environment":"development"}
RawContent      : HTTP/1.1 200 OK
                  Access-Control-Allow-Origin: *
                  Connection: keep-alive
                  Keep-Alive: timeout=5
                  Content-Length: 82
                  Content-Type: application/json; charset=utf-8
                  Date: Mon, 04 May 2026 19:43:28 GMT
                  ...
```

Backend предоставляет маршруты:

- /api/auth
- /api/tests
- /api/results
- /api/admin
- /api/health

Готовое решение: <https://test-constructor-frontend.onrender.com>

Что реализовано в итоговой fullstack-системе

Направление	Результат
Тестирование	создание, редактирование, публикация, прохождение и проверка тестов
Вопросы	один ответ, несколько ответов, сопоставление элементов
Роли	студент, преподаватель, администратор с разными правами
Данные	PostgreSQL-модели users, tests, questions, test_results, system_settings
Аналитика	статистика попыток, средний балл, экспорт PDF и Excel
Инфраструктура	Docker Compose, Nginx, env-настройки, Swagger, healthcheck, CI/CD

Вывод

Проект решает задачу автоматизации контроля знаний: преподаватель получает инструмент создания и анализа тестов, студент - удобное прохождение и самопроверку, администратор - управление системой.

Готово к защите и демонстрации

Запуск: Docker Compose
Документация: Swagger/OpenAPI
Проверка: 100% coverage + fuzzing

A decorative graphic on the left side of the slide consists of several overlapping circles. One circle is a solid light blue, while others are outlines or have a gradient fill. They are arranged in a cluster, with some overlapping each other and others partially visible.

Спасибо за внимание